M. Huzaifa Ch
BCS-3H
L250700

Q No. 1 (a)

```
mov ax, 0xFFFF
mov bx, 0x0000
cmp ax, bx
jb below
```

R = 0x00

∵ 0xFFFF - 0x0000 = 0xFFFF
No borrow occurs CF = 0
for jb CF should be 1 so
no jump is taken

(b)

```
mov ax, 0x7FFF
add ax, 0x0000
jo overflow
```

0x7FFF + 0x0000 = 0x7FFF
No overflow occurs bcz it does not change the sign.

(c)

```
mov cx, 00FF
sub cx, 0x0000
jcxz done
```

jcxz checks if register CX == 0 here
CX = 0x00FF so jump is not taken

(d)

```
mov al, 0x00
add al, 0x45
jo overflow
```

∵ 0x00 + 0x45 = 0x45
No overflow occurs. So jump is not taken

(e)

Choosing (b) snippet

```
mov ax, 0x7FFF
mov ax, 0x8000
```

∵ or convert R from 0000 to 0001. add ax, 0x0001 will also result in addition of two positive numbers and generate a negative number causing overflow to be taken.

Q No. 2

```
nums = dw 0x1234, 0x5678
big  = dd 0xAABBCCDD
```

	nums+0	nums+1	nums+2	nums+3	big+0	big+1	big+2	big+3
	34	12	78	56	DD	CC	BB	AA
After mov	34	00	00	DD	CC	CC	99	99

CamScanner

b) Value ax = 0xCCDD
The instruction mov ax,[big] fetches 16 bit word takes lower address to lower oder byte while byte big+1 move to (AH)
(AL)

c) If third variable is placed b/w nums and big, big would move to higher address; mov ax,[big] would still load **0xCCDD** because the label big tracks new location, but mov [num+3],ax would overwrite bytes of new variable instead of overwriting big+0.

QNo-3 (a) AL=(10110000)$_2$

1) clc
al = 10110000
CF = 0

2) rcl al,1
AL = 0110 0000
CF = 1
(Bit 7 moves to CF)

3) ror al,2
AL = 0001 1000
CF = 0

4) shl al,1 AL = 0011 0000
CF = 0

5) ror al,3
last bit shifted around (bit 5, which was 1)
CF=1

6) shr al,1
0 enters bit 7
Bit 0 (1) moves into CF
AL = 0100 0000
CF = 1

(b)

Reconstruction of AL value prior to shr al,1 is fully possible. A logical shift right (shr) discards bit 0 by transferring it directly into the Carry Flag while inserting a zero into bit 7. Because the original bit 7 is known to have become bit 6 in result and discarded bit 0 is preserved in CF (CF=1). Shifting AL left by 1 and setting bit 0 to CF restores the exact previous state (0x40<<1) | 1 = 0x81 (1000 0001).

CamScanner